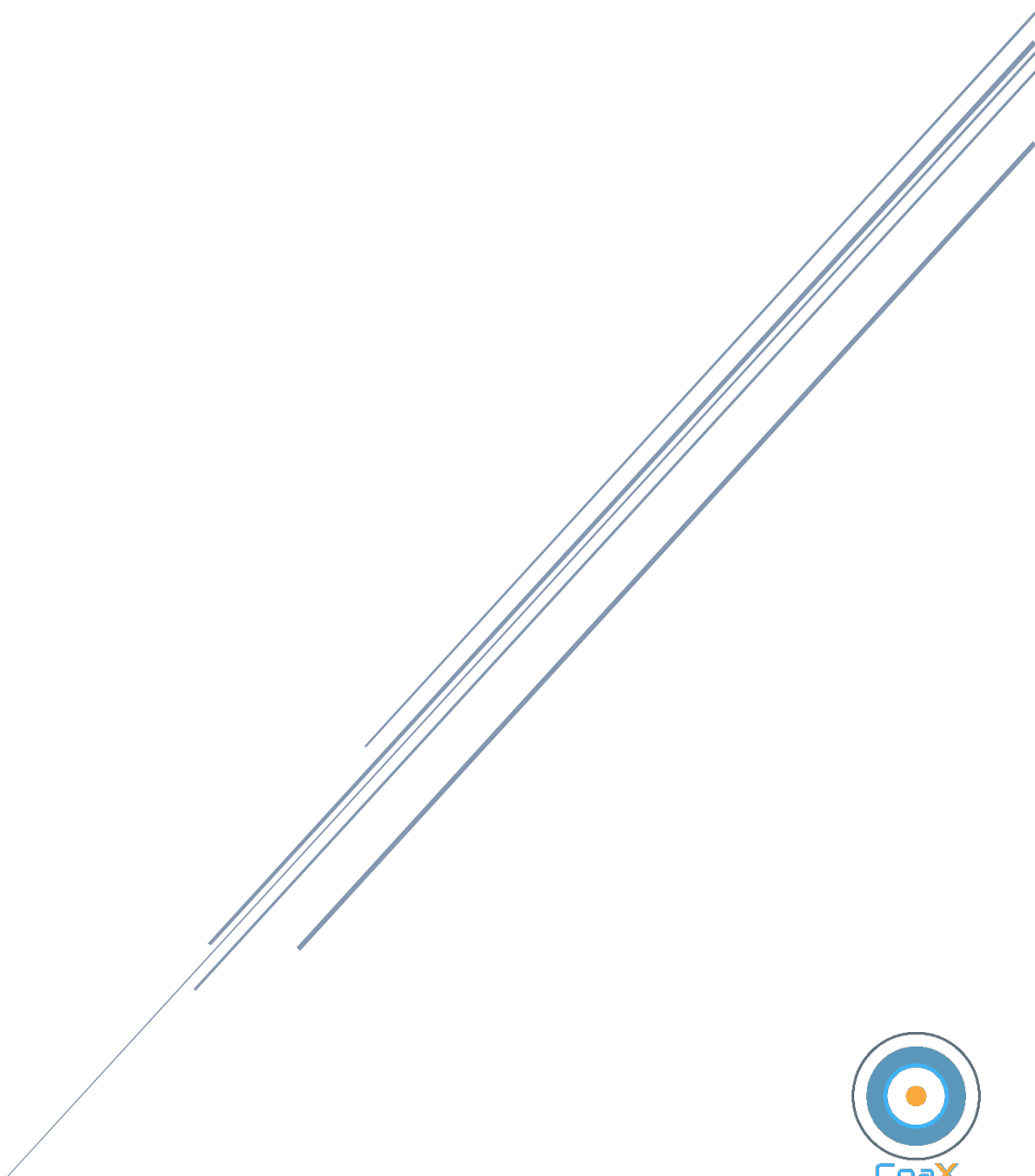


FSL MANAGER

Sistema Web per la Gestione dei percorsi PCTO FSL



26 Maggio 2026
Coax Group

Sommario

Introduzione	2
Descrizione funzionamento	2
Obiettivi implementazione	2
Riferimenti normativi	2
Analisi progetto	3
Utenza	3
Funzioni in base al ruolo	3
Funzioni generali	4
Gestione login ed autenticazione	4
Esportazione dati	5
Descrizione casi d'uso	5
Uso da parte del coordinatore del progetto	5
Uso da parte dello studente	6
Monte ore sviluppo	6
Conclusioni	7

Introduzione

Descrizione funzionamento

FSL Manager è un sistema web per la gestione completa dei percorsi FSL (Formazione Scuola Lavoro) degli studenti. Il sistema nasce dall'esigenza di digitalizzare, centralizzare e semplificare l'intero processo di gestione degli stage scolastici, dalla registrazione delle aziende convenzionate fino al conto finale delle ore svolte. Il progetto è realizzato interamente con tecnologie web standard, senza dipendenze da framework esterni, in modo da garantire la massima portabilità e semplicità di deployment. L'interfaccia è composta da più pagine HTML e da script JavaScript. La persistenza dei dati è affidata all'esportazione JSON tramite download dal browser, rendendo il sistema completamente autonomo e utilizzabile senza connessione a internet.

Obiettivi implementazione

- Semplificare la gestione amministrativa per il coordinatore scolastico
- Permettere agli studenti di registrare in autonomia le presenze giornaliere
- Visualizzare il monte ore di ogni studente tramite inserimento manuale delle ore svolte
- Esportare i dati di tirocinio in per elaborazione esterna
- Creare una base solida e scalabile per l'integrazione futura con database relazionale

Riferimenti normativi

Il sistema è stato progettato nel rispetto delle seguenti normative:

- D.Lgs. 15 aprile 2005, n. 77 - Definizione delle norme generali relative all'alternanza scuola-lavoro
- Legge 13 luglio 2015, n. 107 ("La Buona Scuola") - Art. 33-43, istituzione dei PCTO
- D.Lgs. 13 aprile 2017, n. 62 - Valutazione e certificazione delle competenze nel primo ciclo
- Regolamento UE 2016/679 (GDPR) - Protezione dei dati personali, con riferimento ai dati di minori
- Linee guida MIUR per i PCTO - Circolare ministeriale n. 3 del 3 settembre 2018

Nota GDPR

I dati degli studenti (minorenni) sono trattati in conformità al Regolamento Europeo 2016/679 (GDPR). L'accesso è limitato per ruolo: il coordinatore visualizza i dati di tutti gli studenti, mentre lo studente può accedere esclusivamente ai propri. Nessun dato sensibile viene trasmesso a terzi o salvato su server esterni. Il sistema opera interamente in locale, nel browser dell'utente.

Analisi progetto

Utenza

Il sistema prevede due categorie di utenti, ciascuna con privilegi e funzionalità distinte. L'accesso è differenziato per ruolo fin dalla schermata di login.

Ruolo	Descrizione	Accesso
Coordinatore PCTO	Docente referente per i percorsi PCTO dell'istituto	Gestione completa del sistema
Studente	Alunno iscritto a un percorso PCTO	Propri dati, registro presenze personale

Funzioni in base al ruolo

I requisiti funzionali descrivono le operazioni che il sistema deve essere in grado di eseguire. Sono stati definiti in base alle esigenze concrete dei due attori coinvolti.

Requisito	Ruolo
Login con selezione del ruolo	Entrambi
Registrazione delle ore di tirocinio con data e firma presenze	Studente
Visualizzazione dello stato dei documenti	Studente
Inserimento e modifica dei dati dell'azienda ospitante	Coordinatore

Requisito	Ruolo
Inserimento delle competenze acquisite e delle ore per periodo lavorativo	Coordinatore
Esportazione dei dati inseriti in formato JSON tramite download del file	Entrambi

Funzioni generali

I requisiti non funzionali definiscono le caratteristiche qualitative del sistema, indipendentemente dalle singole funzionalità. Sono stati scelti per garantire un'esperienza d'uso ottimale e una manutenzione semplificata.

Categoria	Descrizione
Usabilità	Interfaccia chiara e navigabile, con side bar e dashboard distinte per ruolo
Portabilità	Funziona su qualsiasi browser moderno senza installazione richiesta
Manutenibilità	Codice suddiviso in file separati per pagina e per ruolo, con CSS condiviso
Sicurezza	Accesso differenziato per ruolo; nessun dato trasmesso in rete
Scalabilità	Architettura predisposta per futura integrazione con database relazionale e backend Java

Gestione login ed autenticazione

Il sistema di autenticazione in confronto direttamente le credenziali inserite con valori fissi nel codice. In caso di successo reindirizza immediatamente senza mostrare messaggi.

Esportazione dati

Il sistema non utilizza un server virtuale e nemmeno la memoria del browser. Tutti i dati vengono salvati tramite esportazione in formato JSON: al click su Save, la funzione raccoglie i valori dai campi del form, li serializza e avvia il download automatico di un file sul browser dell'utente.

Categoria	Descrizione
teacher_data.json	Dati azienda (sector, location, maxStudents, referent, history) e competenze (achievement, skill, workingPeriod, hourCount)
User_data.json	Presenze studente (sign, hourDone) e stato documenti (signedAgreement, trainingProject, finalReport)

Descrizione casi d'uso

Uso da parte del coordinatore del progetto

Il coordinatore accede al sistema tramite main.html inserendo le credenziali (teacher / admin). Dopo il login, viene reindirizzato ad admin.html, dove trova una dashboard con due sezioni principali.

Il coordinatore dispone di due pagine separate nella sidebar. Nella pagina Stage il bottone Save chiama companiesInfoCheck(), che valida i campi azienda tramite try/catch e poi chiama saveFile(). Nella pagina Skills il bottone Save chiama courseInfoCheck(), che valida i campi competenze e chiama saveFile(). In entrambi i casi saveFile() legge tutti i campi del form e scarica il file teacher_data.json. Nota: se si salva dalla pagina Skills, i campi della pagina Stage vengono letti vuoti poiché il DOM è nascosto.

- Pagina Stage: inserimento dati azienda ospitante (settore, sede, numero massimo studenti, referente, note storiche)
- Pagina Skills: inserimento competenze acquisite dallo studente (achievement, skill, periodo lavorativo, ore per periodo)
- Bottone Save in ciascuna pagina: valida i campi e scarica il file teacher_data.json con tutti i dati aggregati

Uso da parte dello studente

Lo studente accede tramite `main.html` con le credenziali (`student / user`) e viene reindirizzato a `user.html`, dove trova la propria dashboard personale con due aree compilabili.

Le operazioni principali sono:

- Registrazione presenze: inserimento della data e delle ore svolte tramite la funzione `presence()`
- Gestione documenti: caricamento e aggiornamento dello stato di convenzione firmata, progetto formativo e relazione finale tramite `documentLoad()`

Al click su Salva, vengono eseguite in sequenza le funzioni `presence()`, `documentLoad()` e `saveFile()`.

Le prime due validano i campi tramite blocchi `try/catch`; la terza aggrega tutti i dati in un oggetto JSON e avvia il download del file sul browser dello studente.

Monte ore sviluppo

Il progetto è stato sviluppato nel periodo maggio 2026, nell'arco di circa tre settimane di lavoro distribuito tra i membri del gruppo. La tabella seguente riporta le attività svolte giorno per giorno con il relativo impegno orario per ciascun membro.

Sviluppatore	Ore di lavoro
Alessio	25
Nicola	22
Anhelina	17

Il totale delle ore di lavoro ammonta a circa 50-60 ore distribuite tra i tre componenti del gruppo. Le attività si sono concentrate principalmente sullo sviluppo dell'interfaccia grafica, della logica di login e del sistema di salvataggio dati in formato JSON.

Conclusioni

FSL Manager rappresenta un primo prototipo funzionante di sistema web per la gestione digitale dei percorsi di Formazione Scuola Lavoro. Il progetto ha permesso al gruppo di applicare concretamente i principi della programmazione orientata agli oggetti, della progettazione di interfacce utente e della gestione dei dati lato client.

I principali risultati raggiunti sono:

- Realizzazione di un sistema di autenticazione con ruoli distinti (studente e coordinatore)
- Sviluppo di dashboard separate e funzionali per ciascun ruolo
- Implementazione del salvataggio dati tramite esportazione JSON
- Struttura del codice modulare e predisposta per futura scalabilità
- Documentazione tecnica completa a corredo del progetto

Punti di miglioramento e sviluppi futuri:

- Integrazione con un backend reale (es. Java Spring Boot) e un database relazionale, sostituendo l'attuale meccanismo di esportazione JSON lato client
- Sistema di autenticazione sicuro con hashing delle password e gestione delle sessioni lato server
- Completamento della logica di validazione nelle funzioni `presence()` e `documentLoad()`, che attualmente leggono i campi ma non implementano controlli sul formato o sulla coerenza dei dati
- Implementazione di notifiche automatiche per scadenze documentali e ritardi nel monte ore
- Pannello di reportistica con grafici di avanzamento per classe

Il progetto ha rispettato i tempi previsti e gli obiettivi formativi definiti nella fase di analisi. L'esperienza ha consolidato le competenze del gruppo in ambito di sviluppo web front-end, progettazione OOP e lavoro collaborativo tramite repository condiviso su GitHub.

Grazie per la lettura

CoaX group